

short or fewer cells in them. They are processed by the `insertionSort()` method shown in [Listing 7-8](#).

If you compare these two methods to their counterparts in the earlier implementation shown in [Listing 7-5](#), they are similar. The small changes do not reduce (nor significantly increase) the complexity of the program. They do, however, result in a big impact on the performance.

[Listing 7-8](#) shows the helper methods used by `quicksort()`. The `medianOfThree()` method performs a three-element sort at the low, middle, and high indices of the subarray. Because the number of items to sort is known and small, it's better to write a few `if` statements to swap values than to make a loop-based sorting routine. After computing the index of the middle item, `mid`, the first two `if` statements swap items to place the item with the lowest key at the `lo` position. Normally for a sort, the next step would be to compare the other two items and ensure the item with the highest key is placed at `hi`. Because this method is preparing the data for partitioning, however, the item with the middle key is placed at `hi`. It returns that item because it is the median of the three keys.

Listing 7-8 *The Helper Methods for `quicksort()`*

```
def medianOfThree(                # Find median of lo, middle, and hi
    self, lo, hi,                 # keys in subarray and put median
    key=identity):                # in hi position for partition
    mid = (lo + hi) // 2          # Compute middle index
    if key(self.get(lo)) > key(self.get(mid)): # Compare 1st pair
        self.swap(lo, mid)       # of keys and swap if lo > mid
    if key(self.get(lo)) > key(self.get(hi)): # Compare 2nd pair
        self.swap(lo, hi)        # of keys and swap if hi is lowest
    # At this point lo has the minimum of the 3 keys
    if key(self.get(hi)) > key(self.get(mid)): # Compare 3rd pair
        self.swap(hi, mid)       # of keys again and swap if hi > mid
    return self.get(hi)           # Return item with median key (@ hi)

def insertionSort(                # Sort subarray by repeated inserts
    self,                        # This insertion sort will be used
    lo=0,                        # on small subarrays by quicksort
    hi=None,
    key=identity):
    if hi is None:                # Fill in hi value if not specified
        hi = len(self) - 1       # as last item in array
    for outer in range(lo + 1, hi + 1): # Mark one item
```

```
temp = self.get(outer) # Store marked item in temp
temp_key = key(temp)
inner = outer          # Inner loop starts at mark at right
while (inner > lo and # If inner hasn't reached lo and next
      temp_key < key(self.get(inner-1))): # item's key is
    self.set(inner, self.get(inner-1)) # smaller, then shift
    inner -= 1                       # next item to right & move inner left
self.set(inner, temp) # Move marked item to 'hole'
```

For subarrays longer than `short`, the median-of-three algorithm is used to select the pivot item. As before, the pivot value (with the median key) is placed in the array at the `hi` index so that the subarray below it can be partitioned. The call to `__part()` inside `quicksort()` (refer to [Listing 7-7](#)) runs on the subarray from `lo + 1` to `hi - 1` because the item with the lowest key from the median-of-three algorithm is stored at `lo` and, hence, does not need to be swapped with any other items during partitioning.

After partitioning, the pivot is swapped into its final position at the lowest index of the higher partition. The recursive calls sort the lower and higher partitions. These include the cells at both ends, `lo` and `hi`, because they now contain items that are in the correct partition but may not be in their final sorted position.

The other helper method is `insertionSort()`. This is the same as the method presented in [Chapter 3](#) but has been adapted to work on subarrays. The now-familiar `lo` and `hi` indices become new parameters and default to the beginning and ending cells of the array. There is a small change in handling the `hi` limit because it indexes the last cell of the subarray instead of one past the last cell.

Quicksort in the AdvancedSorting Visualization Tool

The AdvancedSorting Visualization tool demonstrates the quicksort algorithm using median-of-three pivot selection when the check box is selected. Before, you ran quicksort without checking that box, and it selected the rightmost key as the pivot value. For three or fewer cells, the tool simply sorts the subarray using the insertion sort, regardless of whether pivots are chosen by the median-of-three.

Repeat the experiment of sorting a new 10- to 20-element array filled with increasing keys, but this time check the “Use median of 3” box. Using the rightmost cell’s key caused quicksort to make 17 calls to `__part()` for the 20-element array, but when you use the median-of-three, that drops to 7. No longer

is every subarray partitioned into 1 cell and $N-1$ cells; instead, the subarrays are partitioned roughly in half until they reach three or fewer cells.

Other than this improvement for ordered data, the difference in choosing the pivot produces similar results. It is no faster when sorting random data; its advantages become evident only when sorting ordered data.

Removing Recursion

Another embellishment recommended by many writers is removing recursion from the quicksort algorithm. This task involves rewriting the algorithm to store deferred subarray bounds (`lo` and `hi`) on a stack, and using a loop instead of recursion to oversee the partitioning of smaller and smaller subarrays. The idea in doing this is to speed up the program by removing method calls. This idea, however, arose with older compilers and computer architectures, which imposed a large time penalty for each method call. It's not clear that removing recursion is much of an improvement for modern systems, which handle method calls more efficiently. The depth of recursion could be an issue on systems that limit the size of the call stack. Those systems might allow a larger data stack than the call stack, so the loop-based approach managing the subarray bounds could be able to handle larger arrays without running out of space.

Efficiency of Quicksort

We've said that quicksort operates in $O(N \times \log N)$ time. As you saw in the discussion of mergesort in [Chapter 6](#), this is generally true of the divide-and-conquer algorithms, in which a recursive method divides a range of items into two groups and then calls itself to handle each group. In this situation the logarithm actually has a base of 2: the running time is proportional to $N \times \log_2 N$.

To see why this running time is a good model, let's look at the sequence of subarrays (or partitions) processed by `quicksort()` as visualized in [Figure 7-15](#). The first call is shown as the thickest middle line of the figure and spans all N items in the array. That middle line is numbered 1 at the left to show it is the first call. After partitioning, that subarray is split in two partitions. The index of the partition's pivot is represented by the dotted vertical line. That line connects to two thinner horizontal lines in a different color representing the work done on the lower and higher partitions. The lower one is processed first

in call #2 (lower in terms of array indices and key values; higher in the figure). The higher partition is processed after all the recursive calls needed to handle the lower partition are completed; that is call #9. The process repeats for each recursive level (or depth), splitting the subarray into two smaller subarrays. The partitions at a particular depth all have the same color, while the different depths have different colors in the figure.



Figure 7-15 *Lines corresponding to partitions in quicksort*

The first call to `quicksort()` can be considered depth 1 of the recursive calls. It operates on the full N item array. The two calls at depth 2 also process a total of N items; dealing with them in two separate subarrays. Technically, they only process $N - 1$ items because the pivot of call #1 is not included in either of the depth 2 call's subarrays. Determining the pivot, however, requires three comparisons and zero to three swaps, which is close to the amount of work done if the pivot item were included (one more comparison and perhaps a swap). For determining the total amount of work performed at a particular depth, it's a simplification to say it processes N items.

At depth 3 there are four calls: two from each of the two subarrays at depth 2. Again, the combined items from all four calls totals N (technically $N - 3$ items due to the removed pivots). Similarly, depth 4 has 8 subarrays that cover all N items (technically $N - 7$). The pattern is that at depth D , 2^{D-1} calls cover N items (technically $N - (2^{D-1} - 1)$).

How deep is the recursion? If the choice of pivot were perfect every time, it would split each subarray into equal-sized partitions. That would mean there would be $\log_2 N$ depths (rounded up to the next integer). As we've pointed out, the median-of-three algorithm is only a guess at the true median key value. In

the worst case, the median-of-three could end up putting $N - 2$ items in one partition and 1 item in the other, excluding the pivot item. If all the pivot choices were that bad, the recursive depth would go up to $N/2$ calls, because only 2 items would be excluded at each depth. It's the recursive depth that drives how much memory quicksort uses. It needs to record the subarray ranges for each recursive call on the stack. That means it will need $O(\log N)$ space in the best case and $O(N)$ in the worst.

At each recursive depth, N items are processed, and you saw in the discussion of the partitioning algorithm that it performs $O(N)$ comparisons and a smaller number of swaps (at most $N/2$). Multiplying the work done at each depth with total number of depths shows that the quicksort is $O(N \times \log N)$ in the best case and $O(N \times N) = O(N^2)$ in the worst case.

To keep the quicksort fast, near the optimal $O(N \times \log N)$ complexity, the choice of pivot and the other optimizations are essential. Going back to the partitioning algorithm, this includes the choice of how to handle items with keys equal to the pivot. By choosing to stop advancing `lo` and `hi` when they reach an item with key equal to the pivot and then swapping those items, you increase the chance of some unnecessary swaps but improve the chance of balancing the sizes of the two partitions. In the special case (also called *degenerate case*) of an array of items all having equal keys, this algorithm divides the subarray in half. Keeping the partition sizes close to the optimum of half the subarray lowers the chance of getting the worst-case $O(N^2)$ performance.

Radix Sort

We close this chapter by briefly mentioning a few other different approaches to sorting. The sorts we've looked at so far treat the key as a simple value that is compared with other values to sort the data. The comparison works with multiple data types for the keys such as numbers and strings. The radix sort decomposes the key into digits or characters and arranges the data items according to the value of the digits. Amazingly, no comparisons are necessary. All that's needed is a finite set of possible values for each digit or character. Note that the radix sort is different than the similar but somewhat more complex *radix-exchange sort*.

Algorithm for the Radix Sort

We introduce the radix sort in terms of normal base-10 arithmetic on numeric keys, which is easy to visualize. Efficient implementations of the radix sort using integer keys, however, would use base-2 arithmetic to take advantage of the computer's speed in bit manipulation.

The word **radix** means the base of a system of numbers. Ten is the radix of the decimal system, and two is the radix of the binary system. The **radix sort** involves examining each digit of the key separately, starting with the 1s (least significant) digit.

1. All the data items are divided into 10 groups, according to the value of the 1s digit of their key.
2. These 10 groups are then reassembled into a single group: All the items with keys ending with 0 go first, followed by all the items with keys ending in 1, and so on, up to 9. These reassembled groups are called a *sub-sort*.
3. In the second sub-sort, all data is divided into 10 groups again, but this time according to the value of the 10s digit of their keys. This must be done without changing the order of the previous sort. That is, within each of the 10 groups, the ordering of the items remains the same as it was after step 2. In other words, the sub-sorts must be *stable*.
4. Again the 10 groups are recombined, those with a key whose 10s digit is 0 first, then those with a 10s digit of 1, and so on up to 9.
5. This process is repeated for the remaining digits of the keys. If some keys have fewer digits than others, their higher-order digits are considered to be 0.

Here's an example, using seven data items, each with three digits. Leading zeros are shown for clarity.

```
421 240 035 532 305 430 124           // unsorted array
(240 430) (421) (532) (124) (035 305)   // sorted on 1s digit
(305) (421 124) (430 532 035) (240)     // sorted on 10s digit
(035) (124) (240) (305) (421 430) (532) // sorted on 100s digit
035 124 240 305 421 430 532           // sorted array
```

The parentheses delineate the groups. Within each group, the digits in the appropriate position are the same. To convince yourself that this approach

really works, try it on a piece of paper with some numbers you make up.

Designing a Radix Sort Program

In practice, the original data probably starts out in an ordinary array. Where should the 10 groups go? There's a problem with using another array or an array of 10 arrays. It's not likely there will be exactly the same number of items that have 0 for the first digit as the items having 1 for the first digit, 2 for the first digit, and so on. That makes it hard to know how big to make the arrays for each radix value. The brute-force approach would be to allocate 10 arrays of size N so that all the items would fit in one of them. That could be a very large amount of memory, so another way to solve the problem is to use 10 linked lists instead of 10 arrays. Linked lists expand and contract as needed. You can use this approach here.

Regardless of what structure is used to hold the 10 groups, the groups can be reused for each sub-sort. The data copied into them will be copied back to the original array after each digit of the keys is processed. There's no need to allocate more memory for each pass.

An outer loop looks at each digit of the keys in turn. There are two inner loops: the first takes the data from the array and puts it on the lists; the second copies it from the lists back to the array. Choosing the right kind of linked list will make this efficient. To keep the sub-sorts stable, you need the data to come out of each list in the same order it went in. Which kind of linked list makes this easy? We leave the coding details as an exercise.

Efficiency of the Radix Sort

At first glance, the efficiency of the radix sort seems too good to be true. All you do is copy the original data from the array to the lists and back again. That's two copies per item. If there are 10 data items, this is 20 copies for the first digit. You repeat this procedure once for each digit in the keys. If you assume, say, 5-digit keys, then you'll have 20×5 equals 100 copies. If you have 100 data items, there are 200×5 equals 1,000 copies. The number of copies is proportional to the number of data items, which is $O(N)$, the most efficient sorting algorithm we've seen in this book.

Unfortunately, it's generally true that the more data items needing to be sorted, the longer the keys need to be to distinguish them. If you have 10 times as

much data, you may need to add another digit to the key. The number of copies is proportional to the number of data items times *the number of digits in the key*. The number of digits is proportional to the log of the range of key values, so in most situations you're back to $O(N \times \log N)$ efficiency, the same as quicksort.

There are no comparisons, although it takes time to extract each digit from the number. This must be done once for every two copies. It may be, however, that a given computer can do the digit-extraction in binary more quickly than it can do a comparison. Of course, like mergesort, the radix sort uses $O(N)$ memory to hold all the items in the different linked lists. Compare this to quicksort, which performs all of its swapping in the input array, and needs $O(\log N)$ memory for the stack of calls (and $O(N)$ in the degenerate cases).

Generalizing the Radix Sort

If the keys are strings or *sequences* of values (for example tuples in Python), the radix sort can still be applied under some conditions:

- Each element in the sequence has a known, limited number of possible values.
- There is a fast way to convert each possible value in the sequence to an integer.

For the decimal radix sort, the sequence is of digits and the number of possible values is 10. Call the radix R for short. For characters in a string key, the radix depends on the character set being used. For Latin alphabets, this can be limited to 256 possible characters or even 128 ignoring accented characters (for example, limiting the possibilities to just the ASCII characters). For Unicode character sets, R can grow quite large. If only a subset of the full character set can appear in a key, then R is the size of that subset, but the second condition becomes important. There must be a fast way to convert each character to an integer in the range 0 to $R-1$. That's required to be able to find the right linked list in which to place the items based on a character in their key.

As you might guess, the efficiency of a generalized radix sort depends heavily on the size of R and the length of the keys. Having lots of possible values for each element of a key means maintaining lots of storage structures to hold the items during each pass through the N array elements. There are some

techniques to minimize the number of storage structures to just those that appear in the input keys, and we describe some in later chapters. Having long keys adds a lot of passes through N array elements. Suffice it to say that, the radix sort is only really appropriate in a narrow set of circumstances.

Using a Counting Sort

One variation on implementing the radix sort avoids the use of linked list structures. The **counting sort** uses an array of integer values to count how many item keys match a particular digit of the radix. The counting array starts with all its R elements set to 0. A loop through the N input items increments the appropriate cell in the counting array for a particular digit/element of the sort key. The first two rows of [Figure 7-16](#) show this process for the 10s digit of a 15-element array.



Figure 7-16 *Applying the counting sort on the 10s digit*

At the end of the loop, the counting array holds the number of matching keys for the different values of that particular digit. Some elements of the counting array may be zero, meaning that value of the digit did not appear in any key. A second loop through all but the first element of the counting array adds the count to the count on its left. That makes the r^{th} element of the counting array contain the count of all keys that match r or less. In other words, it has the *cumulative count* of matching keys. The third row of [Figure 7-16](#) shows the result of adding the counts. Note the final value in the array is always the total count of all the keys in the input.

The next step in the counting sort makes use of the cumulative counting array to move items from the input array to an output array. The process starts from the right (highest index) cell. In the data of [Figure 7-16](#), the rightmost key is 64 and its 10s digit is a 6. The algorithm looks at the cumulative counting array at index 6, which is 12. That means there were 12 keys whose 10s digit was 6 or less. All those items must go in the first 12 cells of the output array. It decrements the count of the cumulative array at index 6 by 1. That is 11, and it moves the item from the input array, 64, into the output array at index 11.

The process repeats for each remaining item in the input array. The next one in [Figure 7-16](#) is an item with a key of 90. It looks up 9 in the cumulative counting array, finds 15 stored there, decrements it to 14, and places the item at index 14 in the output, which is the final position. The third item to move is one with a key of 50, whose 10s digit is 5, so it is moved to index 7. When it gets to the fourth item to move, the key is 65. The 10s digit is 6, so it looks at index 6 in the cumulative counting array. This time it finds 11 there because it was decremented once before, so it decrements it again and places item 65 at index 10 of the output array. The last row of [Figure 7-16](#) shows the state of the output array after moving the four items.

The counting sort “knows” where to put the items in the output array based on cumulative counting array values. These items are updated with each move so that every item goes to a unique position. After processing the 10s digit, the entire process would need to be repeated on the 1s digit of all the keys. That process would use the output array from the 10s digit process and put its results back in the input array. That’s when items 65 and 64 of [Figure 7-16](#) would be placed in their correct positions. When all of the digits of all of the keys have been processed, the output array holds the sorted items. If there were an odd number of digits in the keys, one more copy operation would be needed to copy the result back to the input array.

The counting sort has the same complexity as the radix sort. Both run in $O(N \times \log N)$ time and takes $O(N)$ memory. More precisely, both algorithms require $O(N + R)$ memory and run in $O(N \times L)$, where R is the radix and L is the length of the keys, and L must generally be $\log N$ or greater. The difference is in what kind of structure is used for the intermediate results.

Timsort

In [Chapter 3](#), you saw that the simple sorting algorithms could get $O(N^2)$ performance using $O(1)$ memory. In [Chapter 6](#), you saw that mergesort could get $O(N \times \log N)$ performance using $O(N)$ memory. Quicksort can get $O(N \times \log N)$ performance using $O(\log N)$ memory in most cases but can degrade to $O(N^2)$ in some cases. Timsort is a balance between some of these extremes.

Tim Peters implemented Timsort for the Python `sort()` method in 2002 based on the mergesort, insertion sort, and some clever ideas to control the memory usage and guarantee $O(N \times \log N)$ performance. Timsort uses half the memory that mergesort needs and has the side benefit of running in $O(N)$ time on sorted input data, like the insertion sort. It is widely used outside of Python as well because of these characteristics.

Timsort is based around finding runs of items in the original array where the keys are either nondecreasing or strictly decreasing. The idea is to iterate over the items in the array until it finds a key that is lower than the key of its predecessor. That marks the end of a nondecreasing run and the beginning of a new run. Then it can look for the next key that is equal to or higher than its predecessor to find the end of the decreasing run. If it finds a decreasing run of two or more items, it swaps pairs of items from the ends like the quicksort partitioning algorithm to make it a nondecreasing run.

[Figure 7-17](#) illustrates the runs that Timsort identifies. The height of each rectangle represents the key for that item. The array starts with an increasing sequence of four items. This group forms run 1, which is a nondecreasing run. The next four items form run 2, where the keys strictly decrease. After that, the next two items form run 3, which is nondecreasing. Adjacent keys with equal values are treated as nondecreasing, which leads to the next three runs being only two items each. Run 11 illustrates where adjacent equal-valued keys are grouped together in the same run.

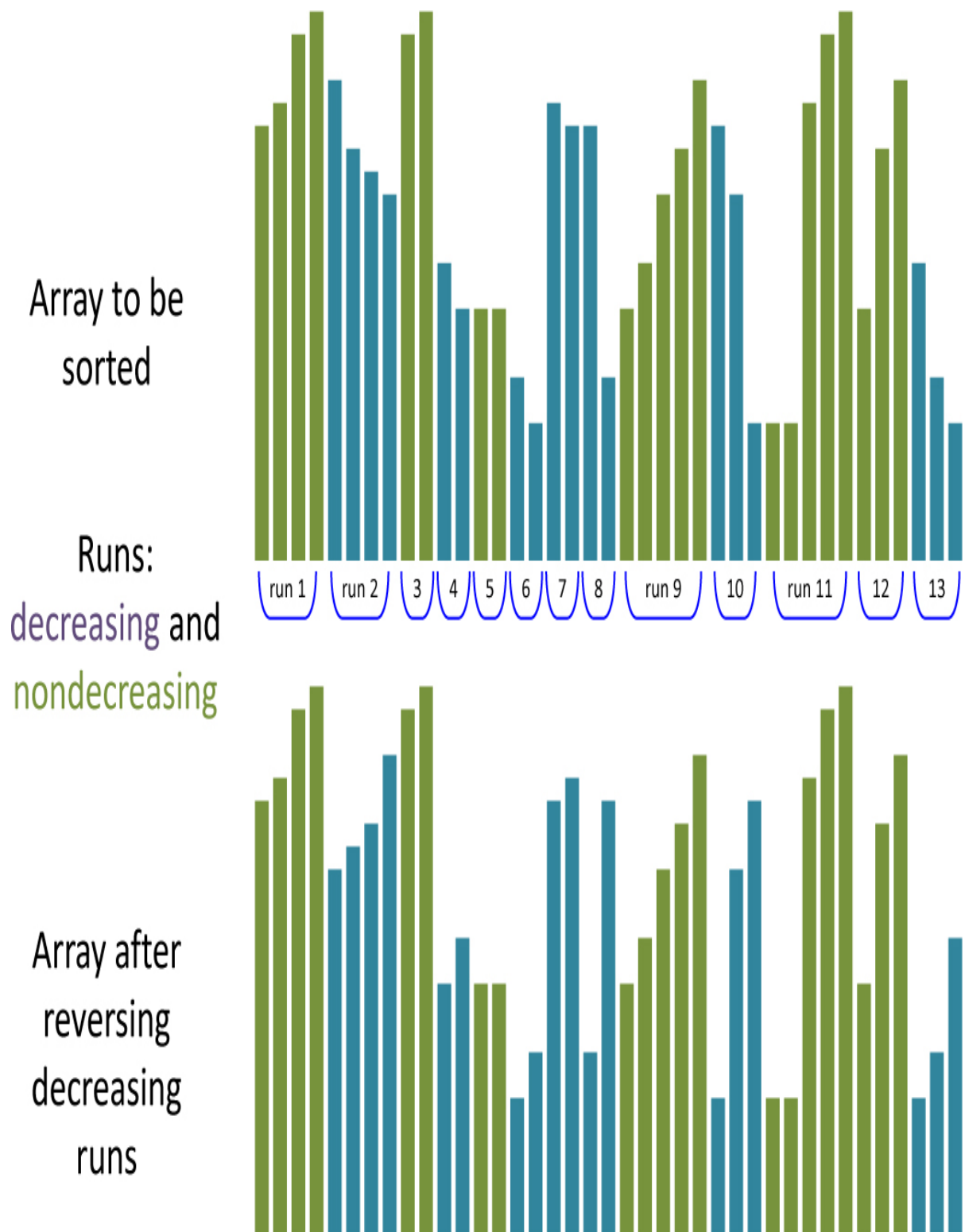


Figure 7-17 *Runs in Timsort*

The bottom panel of [Figure 7-17](#) shows how Timsort reverses all the decreasing runs into nondecreasing ones. Now every run has nondecreasing key values.

It's straightforward to create a generator that yields these runs as it walks along the sequence, converting any decreasing runs into nondecreasing runs, all in $O(N)$ time. A loop in Timsort uses the generator to push the runs on a stack as they are produced. After adding a new run, it checks the conditions of the top 3 or 4 runs on the stack. Under certain conditions, it starts to combine the runs using mergesort. These collapse the small runs into bigger ones. Because the runs have keys that are nondecreasing, they are already sorted. Because it runs mergesort only on adjacent runs, the sorting can be done in place in the input array, leaving a larger sorted run where the two smaller runs were.

By mergesorting the top two runs on the stack repeatedly, the entire stack of runs could be sorted into a single run. Timsort uses several techniques to reduce the amount of extra space needed for those mergesorts. First, it can reduce subarray lengths by finding where in the first run the second run begins and only mergesorting the remainder of the first run. Likewise, items with keys higher than the last key of the first run can be excluded from the second run. When you use binary search, finding where these keys land in the sorted runs is quick, $O(\log N)$.

To keep memory down and maximize the efficiency, Timsort needs to balance the sizes of the runs that it merges. The merge algorithm needs temporary space for a copy of the smaller run, but not for the larger run on the stack. [Figure 7-18](#) illustrates a case where runs K and $K+1$ within the larger input array have been selected for merging. First, Timsort identifies the `left` index, which is the first key higher than the lowest key in run $K+1$ (20). Similarly, the `right` index is the last key lower than the highest key in run K (59). Only the subarray between `left` and `right` needs to be merged because the other items in the full runs are already in their sorted positions.

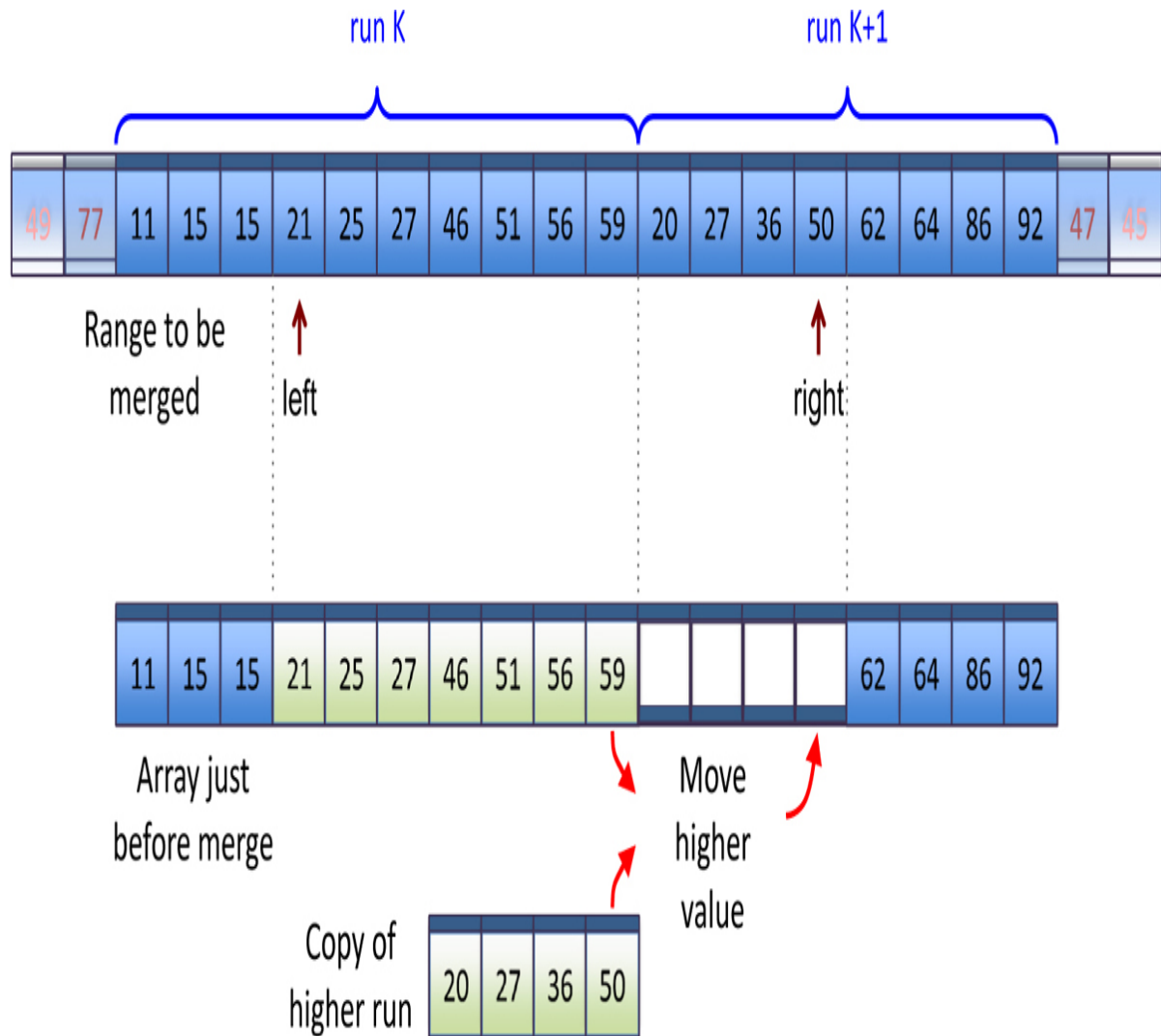


Figure 7-18 *Merging adjacent runs in Timsort*

Knowing `left`, `right`, and the boundary index of the two runs, Timsort finds the length of the sub-runs that need to be merged. In the case shown in the figure, the higher sub-run is shorter, with only four cells. Those items must be copied to a temporary array, as shown in the lower part of the figure. That “empties” the cells in the input array to allow for a merge taking the highest key from the highlighted sub-runs and moving that item to the rightmost empty cell. If the lower sub-run were shorter, then the merge process would move the lowest keyed item into the empty cells at the left (lower) side. The merge process ends when all the items in the copied sub-run have been copied back to the input array.

Efficiency of Timsort

The Timsort algorithm is quite a bit more complex and intricate (perhaps even more beautiful) than quicksort and mergesort. We've left out a few more details, such as using the insertion sort to combine and sort short, adjacent runs and "galloping mode." The benefit of all these techniques is that, overall, it has been shown to operate in $O(N \times \log N)$ time, even in the worst case. In the best case of a forward- or reverse-sorted array, it runs in $O(N)$ time. That extra speed comes at the cost of needing temporary storage for at most $N/2$ cells plus $O(\log N)$ storage for the stack of runs. In certain conditions, such as when the input data is already sorted, no merging of runs is needed, so the memory usage is $O(1)$ in the best case. It ends up needing about half the memory of mergesort to cap the worst-case runtime, delivering the benefit of $O(N)$ time for sorted data.

Summary

- The Shellsort applies the insertion sort to widely spaced elements of an array, then less widely spaced elements, and so on.
- The expression *n-sorting* means sorting every *n*th array element.
- A sequence of numbers, called the *interval sequence*, or *gap sequence*, is used to determine the sorting intervals in the Shellsort.
- A widely used interval sequence is generated by the recursive expression $h_i = 3 \times h_{i-1} + 1$, where the initial value, h_0 , is 1.
- If an array holding 1,000 items were first to be 364-sorted, then 121-sorted, then 40-sorted, then 13-sorted, then 4-sorted, and finally 1-sorted, it would be sorted more efficiently than by simply 1-sorting it.
- The Shellsort is hard to analyze, but runs in approximately $O(N \times (\log N)^2)$ time. This is much faster than the $O(N^2)$ algorithms like the insertion sort, but slower than the $O(N \times \log N)$ algorithms like the quicksort and Timsort.
- To *partition* an array is to divide it into two subarrays, based on a key value of each item.

- The *pivot value* is the value that determines into which group an item will go during partitioning. Items with keys smaller than the pivot value go in the lower group; larger keyed items in the higher group.
- Items with keys equal to the pivot may go in either partition.
- Items in the lower partition have lower indices than those of the higher partition.
- In the partitioning algorithm, two array indices, each in its own `while` loop, start at opposite ends of the array and step toward each other, looking for items that need to be swapped.
- When an array index points to an item that needs to be swapped, its `while` loop exits.
- When both `while` loops exit and the indices have met or passed each other, the partition is complete.
- When both `while` loops exit and the indices have not yet met, the items are swapped.
- Partitioning operates in linear $O(N)$ time, making N plus 1 or 2 comparisons and at most $N/2$ swaps.
- The partitioning algorithm may require extra tests in its inner `while` loops to prevent the indices running off the ends of the array.
- Quicksort partitions an array and then calls itself twice recursively to sort the two resulting partitions (stored in subarrays).
- Subarrays of one or fewer elements are already sorted; this is the base case for quicksort.
- The pivot value in quicksort is the key value of a specific item in the partition, called the *pivot*.
- In a simple version of quicksort, the pivot can be the item at the right end of the subarray.
- During the partitioning, the pivot is placed out of the way on the right and is not moved while the rest of the subarray is partitioned.

- Later the pivot is swapped into the space between the two partitions. This is its final sorted position.
- In the simple version of quicksort, performance is $O(N^2)$ for already sorted (or inversely sorted) data.
- An enhanced version of quicksort selects the pivot as the median of the first, middle, and last items in the subarray. This is called *median-of-three* partitioning.
- Median-of-three partitioning effectively eliminates the problem of $O(N^2)$ performance for already sorted data.
- In median-of-three partitioning, the first, middle, and last items are sorted at the same time the median is determined.
- This three-element sort eliminates the need for the end-of-array tests in the inner `while` loops of the partitioning algorithm.
- Quicksort operates in $O(N \times \log N)$ time (except in degenerate cases such as when the simpler version is applied to already-sorted data).
- Subarrays smaller than a certain size (the *cutoff*) can be sorted by a method other than quicksort.
- The insertion sort is commonly used to sort subarrays smaller than the cutoff.
- The *radix sort* is about as fast as quicksort but uses at least $O(N)$ memory to store copies of the data.
- Generalized radix sort can be used to sort items whose keys are sequences of values, such as strings or tuples.
- The efficiency of a radix sort depends on how many values each sequence element can take, which is the *radix* value, and the maximum length of the key sequences.
- Timsort is a hybrid sorting technique that finds and merges *runs* in the input data.

- The runs in Timsort are adjacent items in the input array with either nondecreasing or decreasing key values.
- Timsort merges adjacent runs using an enhanced version of mergesort.
- Timsort runs in $O(N \times \log N)$ time in the worst case and $O(N)$ time for forward- or reverse-sorted input data.
- Timsort uses $O(N)$ memory in the worst case.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. The Shellsort works by
 - a. partitioning the array into two parts.
 - b. swapping adjacent elements.
 - c. dealing with widely separated elements.
 - d. forming increasingly larger shells of elements that are already sorted.
2. If an array has 100 elements, then Knuth's interval sequence algorithm would start with an interval of _____.
3. To transform the insertion sort into the Shellsort, which of the following do you *not* do?
 - a. Substitute h for 1.
 - b. Insert an algorithm for creating gaps of decreasing width.
 - c. Enclose the normal insertion sort in a loop.
 - d. Change the direction of the indices in the inner loop.
4. What could happen when using the Shellsort with an interval sequence created by repeatedly dividing the array size in half?
5. Fill in the Big O values: The speed of the Shellsort is more than _____ but less than _____.
6. Partitioning is

- a. putting all elements larger than a certain value on one end of the array.
 - b. dividing an array into (almost) equal sized halves.
 - c. partially sorting parts of an array.
 - d. sorting each half of an array separately.
7. During partitioning, each array element is compared to the _____.
8. In partitioning, if an array element is equal to the answer to question 7,
- a. it is passed over.
 - b. it is passed over or not, depending on the other array element.
 - c. it is placed next to the array element at the right end.
 - d. it is swapped.
9. True or False: In quicksort, the pivot can be an arbitrary element of the array.
10. Assuming larger keys on the right, the value returned by the `__partition()` method is
- a. the element between the left and right subarrays.
 - b. the key value of the element between the left and right subarrays.
 - c. the index of the leftmost element in the right subarray.
 - d. the key value of the leftmost element in the right subarray.
11. Quicksort involves partitioning the original input array and then _____.
12. In the simple version of quicksort, after partitioning, the pivot may be
- a. used to find the median of the array.
 - b. exchanged with an element of the right subarray.
 - c. used as the starting point of the next partition.
 - d. discarded.
13. Median-of-three partitioning is a way of choosing the _____.
14. In quicksort, for an array of N elements, the `__partition()` method will examine each element approximately _____ times.

15. What unfortunate outcome could happen when using quicksort and always choosing the rightmost element of subarrays as the input to the `__partition()` method?
16. True or False: You may speed up quicksort if you stop partitioning when the partition size is 5 or less and finish by using a different sort.
17. Radix sort
 - a. uses linked lists to hold elements whose keys have certain digits.
 - b. divides each element by 10 recursively to find its final sorted position.
 - c. first partitions the array in 10 parts, then rearranges elements in each partition by digits in their keys.
 - d. runs faster than quicksort because it doesn't have to compare entire keys.
18. The memory needed for decimal radix sort of an array of N elements is $O(\text{_____})$.
19. The Timsort algorithm starts by
 - a. h -sorting the original array.
 - b. finding the median key of the input array.
 - c. merging the first and second halves of the array until keys start descending.
 - d. finding runs of elements with nondecreasing and decreasing keys.
20. What other sorting methods are used as part of Timsort?
 - a. Shellsort
 - b. mergesort
 - c. quicksort
 - d. h -sort

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

- 7-A In the `ShellSort()` implementation of [Listing 7-1](#), the number of shifts is counted and returned. Would adding a count of the number of key comparisons be helpful in measuring performance? If so, what would it show? If not, why not?
- 7-B Moving the pivot in quicksort by slicing the array and reinserting it between the subarrays as shown in [Figure 7-8](#) would require a lot of copying of cells. Is there another data structure to use in place of the array that would allow slicing and splicing in constant— $O(1)$ —time? If so, which one and why?
- 7-C The partitioning algorithm can put items with keys equal to that of the pivot in either partition. That means that using partitioning in a sort algorithm like quicksort cannot be stable. Try experimenting with partitioning arrays that have many duplicate keys. Using the AdvancedSorting Visualization tool, make a new array of 10 cells and fill it with random values. Delete the rightmost item five times to clear space. Then choose a value somewhere between the minimum and maximum, insert it five times, and shuffle the array. Before running the partitioning using the repeated key value, predict which partition each copy will end up in. Now run the partitioning to check your prediction. Reshuffle the array and try again. Can you generate a situation that demonstrates that the sort will not be stable? Can you generate an initial situation where the repeated key values are stable after the sort?
- 7-D Judicious choice of the pivot value speeds up the quicksort. Using the AdvancedSorting Visualization tool, make a new array of 10 cells and fill it with random values. Try to sort it using only a sequence of Partition operations where *you* choose the pivot. A very poor choice of the pivot might require 10 partitions to sort 10 items. How few partitions does it take you? Can you beat the quicksort? Of course, the initial ordering affects the number of partitions that will be needed. If the initial order happens to be increasing order, you don't need any partitioning. You can always get the same order of 10 items by restarting the visualization tool.
- 7-E Consider using the quicksort algorithm to rank order teams in a tournament. One team is chosen at random as the “pivot” team. It plays each of the other $N-1$ teams. The teams that beat the pivot team go into bracket A and those that lose go into bracket B. Teams that tie the pivot team would go in alternating brackets (first A, then B, then A, then B).

The pivot team is then placed into the bracket with the fewest teams. Then each bracket conducts its own rank ordering using the same algorithm except that the choice of the pivot team for a bracket must be a team with the fewest number of previous selections as pivot within that bracket. The brackets would get smaller with each iteration until they reach size 1, and the sequence of bracket placements would determine the overall ranking of a team with A-A-A-A being highest, B-B-B-B being lowest, A-B-B-B being in the middle, and so on.

How many games would take place for a whole tournament of 16 teams? How many games would each team play? How does that compare with other tournament competition ladders that you're familiar with? If the exact numbers of games is variable or hard to determine, find the best and worst case numbers.

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

7.1 Extend the `ShellSort.py` module of [Listing 7-1](#) to include the two other interval sequences described in the text, namely dividing the interval by 2.2 and truncating, and Flamig's formula. For both methods the first interval, h , should be calculated as if the preceding value of h was the number of array elements. Set up a program that creates an input array and passes copies of that array to the three Shellsort methods (Knuth's sequence, divide by 2.2, and Flamig's sequence). Compare the number of shifts performed on (a) an array that is already sorted, (b) an array that inversely sorted, and (c) a random ordering. Use a loop to perform the test with arrays of length 95 to 100 elements. Is there a clear method that performs fewer shifts?

7.2 Modify the `quicksort()` and helper methods in [Listings 7-7](#) and [7-8](#) to count the number of item copies and key comparisons they make during a sort and return the counts to the caller (using Python's ability to return multiple values). For the copies, a swap counts as 3 copies. For the comparisons, only count comparisons of key values (not array indices). Use the modified method to show how many copies and comparisons

are made when sorting (a) a forward-sorted array of 50 items, (b) a reverse-sorted array of 50 items, (c) a constant value for all 50 items, and (d) an array of 50 random items. For each of those array styles, show the counts when `short` is 3, 7, and 11 to see the effect of using insertion sort for the shorter subarrays. Use the same random array for all variations of the `short` parameter to make a fair comparison.

- 7.3 In Exercise 3.2 in [Chapter 3](#), we suggested finding the median of a set of data by sorting the data and picking the middle element. You might think using quicksort and picking the middle element would be the fastest way to find the median, but there's an even faster way. The quickselect algorithm uses partitioning to find the median without completely sorting the data.

To see how this works, imagine that you partition the data choosing the rightmost item as the pivot, and, by chance, the pivot happens to end up at the middle element. You're done! All the items to the right of the pivot are larger (or equal), and all the items to the left are smaller (or equal), so if the pivot falls in the exact center of the array, then it's the median. The pivot won't end up in the center very often, but you can end up getting the median by solving a smaller problem: repartitioning the partition that contains the middle element.

Suppose your array has seven elements numbered from 0 to 6. The middle is the element at index 3. If you partition this array and the pivot ends up at 4, then you need to partition again from 0 to 4 (the partition that contains index 3), not 5 to 6. If the next pivot ends up at 2, you need to partition from 2 to 6, not 0 to 1. You continue partitioning the appropriate parts recursively, always checking if the pivot falls on the original middle element at index 3. Eventually, it will, and you're done. Because you need fewer partitions than in quicksort, this algorithm is faster.

The quickselect algorithm generalizes the goal a little more by letting the caller ask for any index in the subarray. So, for example, if you wanted the 90th item among an array of 100 numbers, you would call `quickselect()` with a goal index of 90 (or maybe 89, depending on what exactly is needed). If you choose 90, `quickselect()` would return whatever item would get sorted into the cell at that index by `quicksort()`, while only partially sorting the array. To get the median, the goal index would be 50 for the 100-element array. Extend your

solution to Programming Project 7.2 to implement the `quickselect()` method and make another method, `median()`, to get the middle element of an array. Your implementation should make recursive calls somewhat like those in `quicksort`, but they will only partition each subarray, not completely sort it. Use the rightmost element of the subarray as the pivot, as was done in `qsort()`. The process stops when the goal index is found, not when the array is sorted. As an extension of Programming Project 7.2, your method should also return the number of copies and comparisons made. Use your method to show the median found, the number of copies, and the number of comparisons for the same kinds of inputs as the previous project, namely, (a) a forward-sorted array of 50 items, (b) a reverse-sorted array of 50 items, (c) a constant value for all 50 items, and (d) an array of 50 random items.

- 7.4 Implement a decimal radix sort for arrays as described in the “[Designing a Radix Sort Program](#)” section. It should handle variable amounts of data and variable numbers of digits in the key. You could make the radix a parameter as well (so it can be something other than 10), but it will be hard to see what’s happening unless you develop a routine to print values in different bases. Count the number of item copies made by the program. Show the results for arrays of 50 items whose maximum key is 99 and 999 to see the differences key length makes. Your program will need to determine how many digits are in the keys by examining the inputs and the radix (typically as the first pass through all the items is completed).
- 7.5 Implement a simplified version of the Timsort algorithm described in the “[Timsort](#)” section. Specifically, make a generator that looks for runs in an input array that are either nondecreasing or decreasing. The generator should reverse decreasing runs into increasing runs before it yields the `lo`, `hi` index pair that represents the run. Write a `mergerun()` method that takes two adjacent runs in the input array, uses a binary search to find the positions within the runs that need to be merged, copies the shorter section to a temporary array or other structure, and then merges the items back into the input array. The `Timsort()` method should use the generator to identify runs, one at a time, merging the second run into the first until the entire array is sorted. Show the program working on (a) a forward-sorted array of 50 items, (b) a reverse-sorted array of 50 items, (c) a constant value for all 50 items, and (d) an array of 50 random items.

8. Binary Trees

In This Chapter

- [Why Use Binary Trees?](#)
- [Tree Terminology](#)
- [An Analogy](#)
- [How Do Binary Search Trees Work?](#)
- [Finding a Node](#)
- [Inserting a Node](#)
- [Traversing the Tree](#)
- [Finding Minimum and Maximum Key Values](#)
- [Deleting a Node](#)
- [The Efficiency of Binary Search Trees](#)
- [Trees Represented as Arrays](#)
- [Printing Trees](#)
- [Duplicate Keys](#)
- [The BinarySearchTreeTester.py Program](#)
- [The Huffman Code](#)

In this chapter we switch from algorithms, the focus of [Chapter 7](#), “[Advanced Sorting](#),” to data structures. Binary trees are one of the fundamental data storage structures used in programming. They provide advantages that the data

structures you've seen so far cannot. In this chapter you learn why you would want to use trees, how they work, and how to go about creating them.

Why Use Binary Trees?

Why might you want to use a tree? Usually, because it combines the advantages of two other structures: an ordered array and a linked list. You can search a tree quickly, as you can an ordered array, and you can also insert and delete items quickly, as you can with a linked list. Let's explore these topics a bit before delving into the details of trees.

Slow Insertion in an Ordered Array

Imagine an array in which all the elements are arranged in order—that is, an ordered array—such as you saw in [Chapter 2](#), “[Arrays](#).” As you learned, you can quickly search such an array for a particular value, using a binary search. You check in the center of the array; if the object you're looking for is greater than what you find there, you narrow your search to the top half of the array; if it's less, you narrow your search to the bottom half. Applying this process repeatedly finds the object in $O(\log N)$ time. You can also quickly traverse an ordered array, visiting each object in sorted order.

On the other hand, if you want to insert a new object into an ordered array, you first need to find where the object will go and then move all the objects with greater keys up one space in the array to make room for it. These multiple moves are time-consuming, requiring, on average, moving half the items ($N/2$ moves). Deletion involves the same multiple moves and is thus equally slow.

If you're going to be doing a lot of insertions and deletions, an ordered array is a bad choice.

Slow Searching in a Linked List

As you saw in [Chapter 5](#), “[Linked Lists](#),” you can quickly perform insertions and deletions on a linked list. You can accomplish these operations simply by changing a few references. These two operations require $O(1)$ time (the fastest Big O time).

Unfortunately, however, *finding* a specified element in a linked list is not as fast. You must start at the beginning of the list and visit each element until you find the one you're looking for. Thus, you need to visit an average of $N/2$ objects, comparing each one's key with the desired value. This process is slow, requiring $O(N)$ time. (Notice that times considered fast for a sort are slow for the basic data structure operations of insertion, deletion, and search.)

You might think you could speed things up by using an ordered linked list, in which the elements are arranged in order, but this doesn't help. You still must start at the beginning and visit the elements in order because there's no way to access a given element without following the chain of references to it. You could abandon the search for an element after finding a gap in the ordered sequence where it should have been, so it would save a little time in identifying missing items. Using an ordered list only helps make traversing the nodes in order quicker and doesn't help in finding an arbitrary object.

Trees to the Rescue

It would be nice if there were a data structure with the quick insertion and deletion of a linked list, along with the quick searching of an ordered array. Trees provide both of these characteristics and are also one of the most interesting data structures.

What Is a Tree?

A tree consists of **nodes** connected by **edges**. [Figure 8-1](#) shows a tree. In such a picture of a tree the nodes are represented as circles, and the edges as lines connecting the circles.

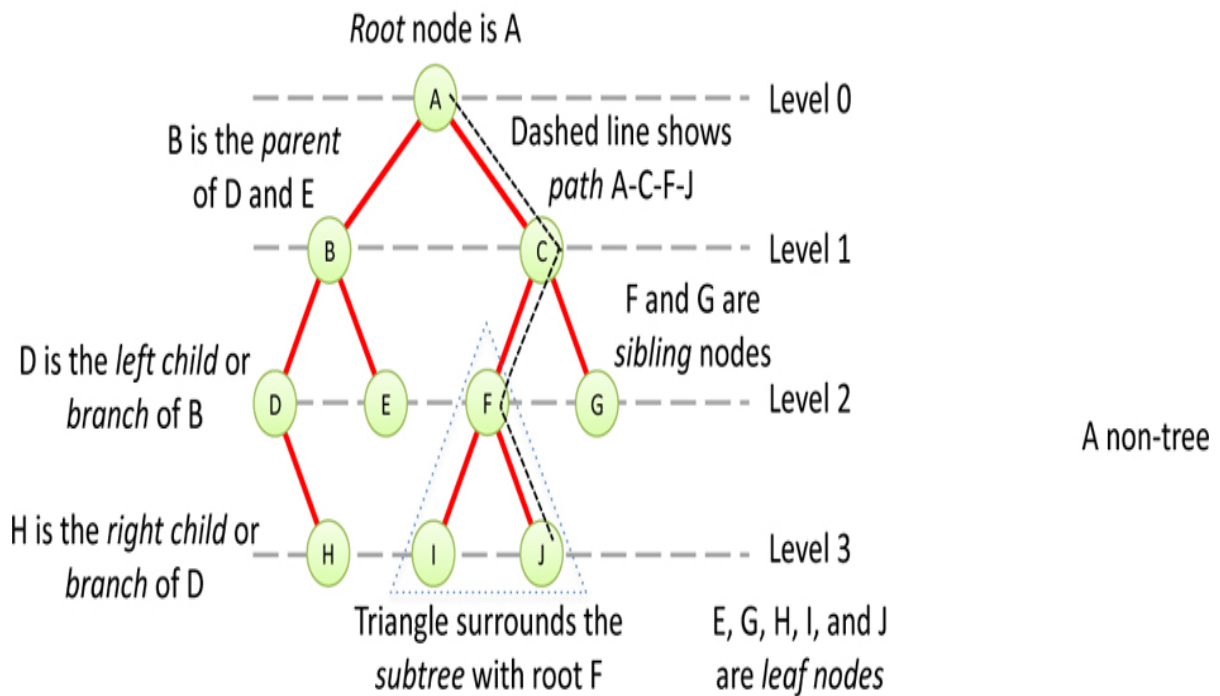
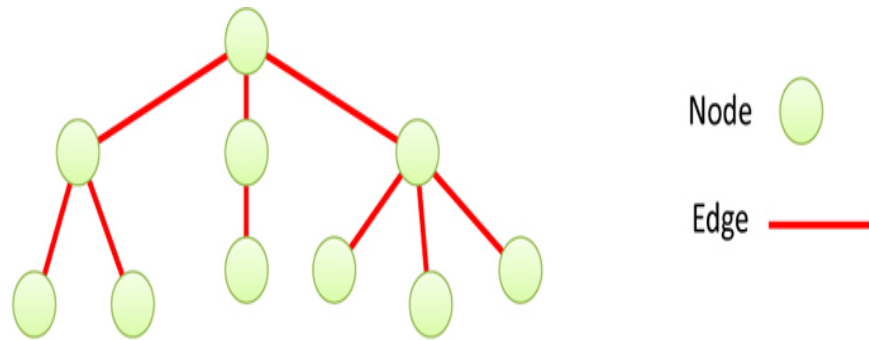


Figure 8-1 *A general (nonbinary) tree*

Trees have been studied extensively as abstract mathematical entities, so there's a large amount of theoretical knowledge about them. A tree is actually an instance of a more general category called a **graph**. The types and arrangement of edges connecting the nodes distinguish trees and graphs, but you don't need to worry about the extra issues graphs present. We discuss graphs in [Chapter 14, "Graphs,"](#) and [Chapter 15, "Weighted Graphs."](#)

In computer programs, nodes often represent entities such as file folders, files, departments, people, and so on—in other words, the typical records and items

stored in any kind of data structure. In an object-oriented programming language, the nodes are objects that represent entities, sometimes in the real world.

The lines (edges) between the nodes represent the way the nodes are related. Roughly speaking, the lines represent convenience: it's easy (and fast) for a program to get from one node to another if a line connects them. In fact, the *only* way to get from node to node is to follow a path along the lines. These are essentially the same as the references you saw in linked lists; each node can have some references to other nodes. Algorithms are restricted to going in one direction along edges: from the node with the reference to some other node. Doubly linked nodes are sometimes used to go both directions.

Typically, one node is designated as the **root** of the tree. Just like the head of a linked list, all the other nodes are reached by following edges from the root. The root node is typically drawn at the top of a diagram, like the one in [Figure 8-1](#). The other nodes are shown below it, and the further down in the diagram, the more edges need to be followed to get to another node. Thus, tree diagrams are small on the top and large on the bottom. This configuration may seem upside-down compared with real trees, at least compared to the parts of real trees above ground; the diagrams are more like tree root systems in a visual sense. This arrangement makes them more like charts used to show family trees with ancestors at the top and descendants below. Generally, programs start an operation at the small part of the tree, the root, and follow the edges out to the broader fringe. It's (arguably) more natural to think about going from top to bottom, as in reading text, so having the other nodes below the root helps indicate the relative order of the nodes.

There are different kinds of trees, distinguished by the number and type of edges. The tree shown in [Figure 8-1](#) has more than two children per node. (We explain what "children" means in a moment.) In this chapter we discuss a specialized form of tree called a **binary tree**. Each node in a binary tree has a maximum of two children. More general trees, in which nodes can have more than two children, are called **multiway trees**. We show examples of multiway trees in [Chapter 9](#), "[2-3-4 Trees and External Storage](#)."

Tree Terminology

Many terms are used to describe particular aspects of trees. You need to know them so that this discussion is comprehensible. Fortunately, most of these terms

are related to real-world trees or to family relationships, so they're not hard to remember. [Figure 8-2](#) shows many of these terms applied to a binary tree.



Figure 8-2 *Tree terms*

Root

The node at the top of the tree is called the **root**. There is only one root in a tree, labeled A in the figure.

Path

Think of someone walking from node to node along the edges that connect them. The resulting sequence of nodes is called a **path**. For a collection of nodes and edges to be defined as a tree, there must be one (and only one!) path from the root to any other node. [Figure 8-3](#) shows a nontree. You can see that it violates this rule because there are multiple paths from A to nodes E and F. This is an example of a *graph* that is not a tree.



Figure 8-3 *A nontree*

Parent

Any node (except the root) has exactly one edge running upward to another node. The node above it is called the **parent** of the node. The root node must not have a parent.

Child

Any node may have one or more lines running downward to other nodes. These nodes below a given node are called its **children**, or sometimes, **branches**.

Sibling

Any node other than the root node may have **sibling** nodes. These nodes have a common parent node.

Leaf

A node that has no children is called a **leaf node** or simply a **leaf**. There can be only one root in a tree, but there can be many leaves. In contrast, a node that has children is an **internal node**.

Subtree

Any node (other than the root) may be considered to be the root of a **subtree**, which consists of its children, and its children's children, and so on. If you think in terms of families, a node's subtree contains all its **descendants**.

Visiting

A node is **visited** when program control arrives at the node, usually for the purpose of carrying out some operation on the node, such as checking the value of one of its data fields or displaying it. Merely passing over a node on the path from one node to another is not considered to be visiting the node.

Traversing

To **traverse** a tree means to visit all the nodes in some specified order. For example, you might visit all the nodes in order of ascending key value. There are other ways to traverse a tree, as we'll describe later.

Levels

The **level** of a particular node refers to how many generations the node is from the root. If you assume the root is Level 0, then its children are at Level 1, its grandchildren are at Level 2, and so on. This is also sometimes called the **depth** of a node.

Keys

You've seen that one data field in an object is usually designated as a **key value**, or simply a **key**. This value is used to search for the item or perform other operations on it. In tree diagrams, when a circle represents a node holding a data item, the key value of the item is typically shown in the circle.

Binary Trees

If every node in a tree has at most two children, the tree is called a **binary tree**. In this chapter we focus on binary trees because they are the simplest and the most common.

The two children of each node in a binary tree are called the **left child** and the **right child**, corresponding to their positions when you draw a picture of a tree, as shown in [Figure 8-2](#). A node in a binary tree doesn't necessarily have the maximum of two children; it may have only a left child or only a right child, or it can have no children at all (in which case it's a leaf).

Binary Search Trees

The kind of binary tree we discuss at the beginning of this chapter is technically called a **binary search tree**. The keys of the nodes have a particular ordering in search trees. [Figure 8-4](#) shows a binary search tree.

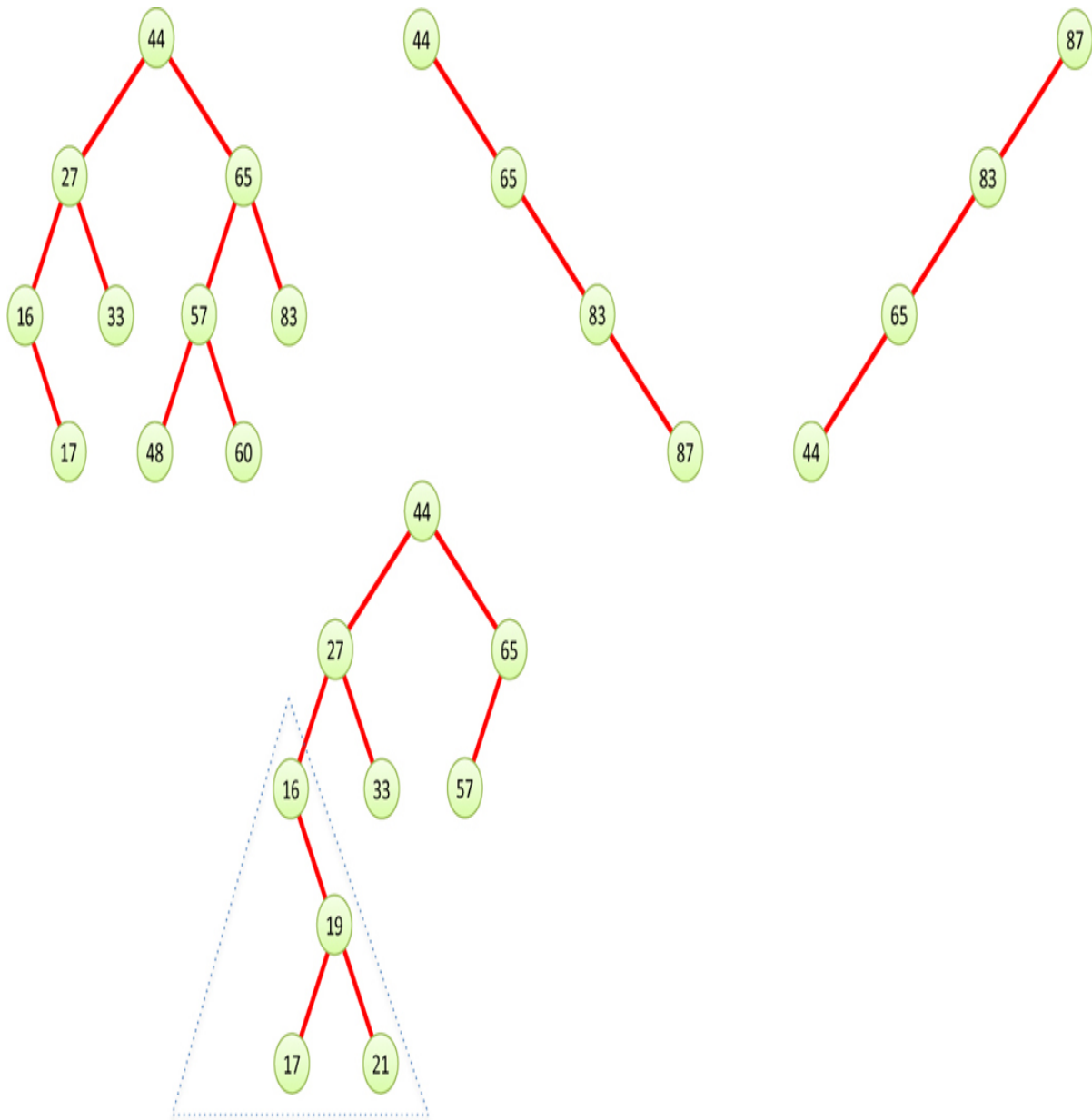


Figure 8-4 *A binary search tree*

Note

The defining characteristic of a binary search tree is this: a node's left child must have a key less than its parent's key, and a node's right child must have a key greater than or equal to that of its parent.

An Analogy

One commonly encountered tree is the hierarchical file system on desktop computers. This system was modeled on the prevailing document storage technology used by businesses in the twentieth century: filing cabinets containing folders that in turn contained subfolders, down to individual documents. Computer operating systems mimic that by having files stored in a hierarchy. At the top of the hierarchy is the root directory. That directory contains “folders,” which are subdirectories, and files, which are like the paper documents. Each subdirectory can have subdirectories of its own and more files. These all have analogies in the tree: the root directory is the root node, subdirectories are nodes with children, and files are leaf nodes.

To specify a particular file in a file system, you use the full path from the root directory down to the file. This is the same as the path to a node of a tree. Uniform resource locators (URLs) use a similar construction to show a path to a resource on the Internet. Both file system pathnames and URLs allow for many levels of subdirectories. The last name in a file system path is either a subdirectory or a file. Files represent leaves; they have no children of their own.

Clearly, a hierarchical file system is not a binary tree because a directory may have many children. A hierarchical file system differs in another significant way from the trees that we discuss here. In the file system, subdirectories contain no data other than attributes like their name; they contain only references to other subdirectories or to files. Only files contain data. In a tree, every node contains data. The exact type of data depends on what’s being represented: records about personnel, records about components used to construct a vehicle, and so forth. In addition to the data, all nodes except leaves contain references to other nodes.

Hierarchical file systems differ from binary search trees in other aspects, too. The purpose of the file system is to organize files; the purpose of a binary search tree is more general and abstract. It’s a data structure that provides the common operations of insertion, deletion, search, and traversal on a collection of items, organizing them by their keys to speed up the operations. The analogy between the two is meant to show another familiar system that shares some important characteristics, but not all.

How Do Binary Search Trees Work?

Let's see how to carry out the common binary tree operations of finding a node with a given key, inserting a new node, traversing the tree, and deleting a node. For each of these operations, we first show how to use the Binary Search Tree Visualization tool to carry it out; then we look at the corresponding Python code.

The Binary Search Tree Visualization Tool

For this example, start the Binary Search Tree Visualization tool (the program is called `BinaryTree.py`). You should see a screen something like that shown in [Figure 8-5](#).



Figure 8-5 *The Binary Search Tree Visualization tool*

Using the Visualization Tool

The key values shown in the nodes range from 0 to 99. Of course, in a real tree, there would probably be a larger range of key values. For example, if telephone numbers were used for key values, they could range up to 999,999,999,999,999 (15 digits including country codes in the International Telecommunication Union standard). We focus on a simpler set of possible keys.

Another difference between the visualization tool and a real tree is that the visualization tool limits its tree to a depth of five; that is, there can be no more than five levels from the root to the bottom (level 0 through level 4). This restriction ensures that all the nodes in the tree will be visible on the screen. In a real tree the number of levels is unlimited (until the computer runs out of memory).

Using the visualization tool, you can create a new tree whenever you want. To do this, enter a number of items and click the Erase & Random Fill button. You can ask to fill with 0 to 99 items. If you choose 0, you will create an empty tree. Using larger numbers will fill in more nodes, but some of the requested nodes may not appear. That's due to the limit on the depth of the tree and the random order the items are inserted. You can experiment by creating trees with different numbers of nodes to see the variety of trees that come out of the random sequencing.

The nodes are created with different colors. The color represents the data stored with the key. We show a little later how that data is updated in some operations.

Constructing Trees

As shown in the visualization tool, the tree's shape depends both on the items it contains as well as the order the items are inserted into the tree. That might seem strange at first. If items are inserted into a sorted array, they always end up in the same order, regardless of their sequencing. Why are binary search trees different?

One of the key features of the binary search tree is that it does not have to fully order the items as they are inserted. When it adds a new item to an existing tree, it decides where to place the new leaf node by comparing its key with that of the nodes already stored in the tree. It follows a path from the root down to a missing child where the new node "belongs." By choosing the left child when the new node's key is less than the key of an internal node and the right child for other values, there will always be a unique path for the new node. That unique path means you can easily find that node by its key later, but it also means that the previously inserted items affect the path to any new item.

For example, if you start with an empty binary search tree and insert nodes in increasing key order, the unique path for each one will always be the rightmost path. Each insertion adds one more node at the bottom right. If you reverse the order of the nodes and insert them into an empty tree, each insertion will add the node at the bottom left because the key is lower than any other in the tree so far. [Figure 8-6](#) shows what happens if you insert nodes with keys 44, 65, 87, and 87 in forward or reverse order.

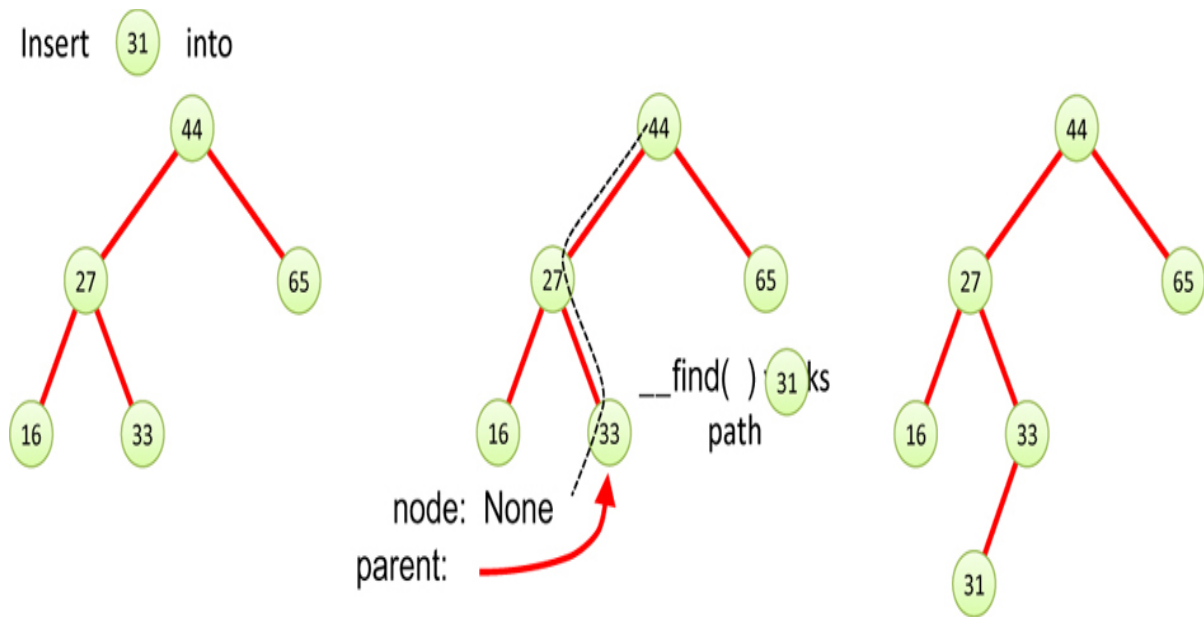


Figure 8.6 *Trees made by inserting nodes in sorted order*

Unbalanced Trees

The trees shown in [Figure 8-6](#), don't look like trees. In fact, they look more like linked lists. One of the goals for a binary search tree is to speed up search for a particular node, so having to step through a linked list to find the node would not be an improvement. To get the benefit of the tree, the nodes should be roughly balanced on both sides of the root. Ideally, each step along the path to find a node should cut the number of nodes to search in half, just like in binary searches of arrays and the guess-a-number game described in [Chapter 2](#).

We can call some trees **unbalanced**; that is, they have most of their nodes on one side of the root or the other, as shown in [Figure 8-7](#). Any subtree may also be unbalanced like the outlined one on the lower left of the figure. Of course, only a fully balanced tree will have equal numbers of nodes on the left and right subtrees (and being fully balanced, every node's subtree will be fully balanced too). Inserting nodes one at a time on randomly chosen items means most trees will be unbalanced by one or more nodes as they are constructed, so you typically cannot expect to find fully balanced trees. In the following chapters, we look more carefully at ways to balance them as nodes are inserted and deleted.



Figure 8-7 *An unbalanced tree (with an unbalanced subtree)*

Trees become unbalanced because of the order in which the data items are inserted. If these key values are inserted randomly, the tree will be more or less balanced. When an ascending sequence (like 11, 18, 33, 42, 65) or a descending sequence is encountered, all the values will be right children (if ascending) or left children (if descending), and the tree will be unbalanced. The key values in the visualization tool are generated randomly, but of course some short ascending or descending sequences will be created anyway, which will lead to local imbalances.

If a tree is created by data items whose key values arrive in random order, the problem of unbalanced trees may not be too much of a problem for larger trees because the chances of a long run of numbers in a sequence is small.

Sometimes, however, key values will arrive in strict sequence; for example, when someone doing data entry arranges a stack of forms into alphabetical order by name before entering the data. When this happens, tree efficiency can be seriously degraded. We discuss unbalanced trees and what to do about them in [Chapters 9](#) and [10](#).

Representing the Tree in Python Code

Let's start implementing a binary search tree in Python. As with other data structures, there are several approaches to representing a tree in the computer's memory. The most common is to store the nodes at (unrelated) locations in memory and connect them using references in each node that point to its children.

You can also represent a tree in memory as an array, with nodes in specific positions stored in corresponding positions in the array. We return to this possibility at the end of this chapter. For our sample Python code we'll use the approach of connecting the nodes using references, similar to the way linked lists were implemented in [Chapter 5](#).

The BinarySearchTree Class

We need a class for the overall tree object: the object that holds, or at least leads to, all the nodes. We'll call this class `BinarySearchTree`. It has only one field, `__root`, that holds the reference to the root node, as shown in [Listing 8-1](#). This is very similar to the `LinkedList` class that was used in [Chapter 5](#) to represent linked lists. The `BinarySearchTree` class doesn't need fields for the other nodes because they are all accessed from the root node by following other references.

Listing 8-1 *The Constructor for the BinarySearchTree Class*

```
class BinarySearchTree(object): # A binary search tree class

    def __init__(self):         # The tree organizes nodes by their
        self.__root = None     # keys. Initially, it is empty.
```

The constructor initializes the reference to the root node as `None` to start with an empty tree. When the first node is inserted, `__root` will point to it as shown in the visualization tool example of [Figure 8-5](#). There are, of course, many methods that operate on `BinarySearchTree` objects, but first, you need to define the nodes inside them.

The __Node Class

The nodes of the tree contain the data representing the objects being stored (contact information in an address book, for example), a key to identify those objects (and to order them), and the references to each of the node's two children. Because callers that create `BinarySearchTree` objects should not directly alter the nodes, we make a private `__Node` class inside that class. [Listing 8-2](#) shows how an internal class can be defined inside the `BinarySearchTree` class.